DevOps Mini Project

Complete Technical Documentation

Project By: Manvhi Pipriya

GitHub: github.com/manvhi/devops-mini-project

Live App: https://devops-app-49qd.onrender.com

Docker Hub: hub.docker.com/r/manvhipipriya/devops-app

Date: April 2026

1. Project Overview

This project demonstrates a complete DevOps pipeline for a Spring Boot Task Manager web application. The goal was not just to build an application, but to automate the entire software delivery process — from writing code to deploying it live on the internet.

The pipeline ensures that every time a developer pushes code to GitHub, the following happens automatically without any human intervention:

- Code is compiled and built using Maven
- Automated tests are executed (JUnit + MockMvc)
- A Docker container image is built
- The image is pushed to Docker Hub
- The live application is redeployed on Render

1.1 Project URLs

Resource	URL
GitHub Repository	https://github.com/manvhi/devops-mini-project
Live Application	https://devops-app-49qd.onrender.com
Docker Hub Image	https://hub.docker.com/r/manvhipipriya/devops-app

2. Technology Stack

Tool / Technology	Purpose	Category
Java 17 + Spring Boot 3.2.5	Backend REST API framework	Application
HTML / CSS / JavaScript	Frontend Task Manager UI	Application
Maven	Build tool, dependency management, test runner	Build
JUnit 5 + MockMvc	Unit and integration testing for API	Testing
Selenium WebDriver	Browser UI automation testing	Testing
Git + GitHub	Version control and source code hosting	Version Control
GitHub Actions	CI/CD pipeline automation	CI/CD
Docker	Containerization of the application	Containerization
Docker Hub	Cloud registry for Docker images	Registry

Render	Cloud platform for live deployment	Deployment

3. Application — Task Manager

The application is a Task Manager built with Spring Boot. It provides a REST API and a browser-based UI for managing tasks.

3.1 REST API Endpoints

Method	Endpoint	Description	Response
GET	/tasks	Returns all tasks	JSON array of tasks
POST	/tasks	Adds a new task	"Task added"
DELETE	/tasks/{index}	Deletes task by index	"Task deleted"

3.2 TaskController.java

The controller handles all API requests. Key annotations used:

- @RestController — marks class as REST API controller
- @CrossOrigin(origins = "*") — allows requests from any origin (CORS fix)
- @GetMapping, @PostMapping, @DeleteMapping — maps HTTP methods to functions
- @PathVariable — extracts index from URL for delete operation

3.3 Frontend UI

A single-page HTML/CSS/JavaScript UI is served from src/main/resources/static/index.html. It communicates with the REST API using the Fetch API to add, display, and delete tasks dynamically without page reload.

4. Project Structure

```
devops-mini-project/
  .github/
    workflows/
      ci.yml              <- GitHub Actions CI/CD pipeline
  devops-app/
    Dockerfile            <- Docker build instructions
    pom.xml               <- Maven build config + dependencies
    src/
```

```
    main/
      java/com/example/devopsdemo/
        DevopsdemoApplication.java   <- Spring Boot entry point
        TaskController.java          <- REST API controller
      resources/
        static/
          index.html    <- Frontend UI
        application.properties
    test/
      java/com/example/devopsdemo/
        DevopsdemoApplicationTests.java  <- JUnit API tests
        SeleniumTest.java                <- Browser UI tests
```

5. Dockerfile — Containerization

A multi-stage Dockerfile is used to keep the final image small and efficient.

5.1 Multi-Stage Build Explained

Stage	Base Image	Purpose	Size
Stage 1 (build)	maven:3.9.6-eclipse-temurin-17	Compiles code and creates JAR	~500MB (temporary)
Stage 2 (run)	eclipse-temurin:17-jre	Runs the JAR file	~200MB (final)

5.2 Dockerfile

```
# Stage 1: Build
FROM maven:3.9.6-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests
```

```
# Stage 2: Run
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

5.3 Why Multi-Stage?

- Stage 1 needs Maven to build — but Maven is not needed at runtime
- Stage 2 only has JRE — much smaller and more secure
- Final image is ~200MB instead of ~500MB
- Industry best practice for production Docker images

6. CI/CD Pipeline — GitHub Actions

The pipeline is defined in .github/workflows/ci.yml and triggers automatically on every push to main, dev, or feature branches.

6.1 Pipeline Flow

Stage	Tool	What Happens	On Failure
Trigger	GitHub Actions	Push to main branch detected	-
Checkout	actions/checkout@v4	Downloads code from GitHub	Pipeline stops
Setup Java	actions/setup-java@v4	Installs Java 17 (Temurin)	Pipeline stops
Compile	Maven	mvn clean compile	Pipeline stops
Test	Maven + JUnit	mvn test (Selenium excluded)	Pipeline stops
Package	Maven	mvn package → creates JAR	Pipeline stops
Docker Build	Docker	Builds image from Dockerfile	Pipeline stops
Docker Push	Docker Hub	Pushes image to registry	Pipeline stops
Deploy	Render	Auto-redeploys live app	Alert only

6.2 Two Jobs — Build and Docker

The pipeline has 2 separate jobs:

- Job 1 (build): Compiles code, runs tests, creates JAR, uploads as artifact
- Job 2 (docker): Only runs if Job 1 succeeds — builds and pushes Docker image

The 'needs: build' keyword ensures Docker job never runs if tests fail. This prevents broken code from reaching production.

6.3 Secrets Used

Secret Name	Value	Used For
DOCKER_USERNAME	manvhipipriya	Docker Hub login
DOCKER_PASSWORD	Docker Access Token	Docker Hub authentication

7. Testing Strategy

Two levels of testing are implemented to verify both the API layer and the UI layer.

7.1 JUnit + MockMvc Tests (API Testing)

Test Name	What It Tests	Expected Result
testGetTasksReturnsEmptyList	GET /tasks on fresh start	Returns empty array []
testAddTask	POST /tasks with a task string	Returns "Task added"
testGetTasksAfterAdding	POST then GET /tasks	Task appears in list

Key annotations:

- @SpringBootTest — loads full Spring application context
- @AutoConfigureMockMvc — injects MockMvc for simulating HTTP requests
- mockMvc.perform() — simulates GET/POST/DELETE without real HTTP server

7.2 Selenium UI Tests (Browser Testing)

Selenium automates a real Chrome browser to test the UI as a real user would.

Action	What Selenium Does	Verification
Open App	Navigates to localhost:8080	Page loads successfully
Add Task	Types in input, clicks Add button	Task appears in list
Verify Add	Shows alert popup	"Task Added Successfully!"

Delete Task	Clicks Delete button	Task removed from list
Verify Delete	Shows alert popup	"Task Deleted Successfully!"

7.3 Why Selenium is Excluded from CI

Selenium requires a real browser with a display. GitHub Actions runners run in a headless Linux environment with no display. Therefore Selenium is excluded in CI using:

```
mvn -Dtest=!SeleniumTest test
```

In a production setup, headless Chrome would be configured in CI to run Selenium tests automatically.

8. Deployment — Render

The application is deployed on Render's free tier using Docker.

8.1 How Render Works

- Render watches the GitHub repository for new commits
- On every push to main, Render automatically pulls the latest Docker image
- Render stops the old container and starts a new one with the updated image
- The live URL is updated within 3-5 minutes of a push

8.2 Render Configuration

Setting	Value
Service Type	Web Service
Language	Docker
Branch	main
Root Directory	devops-app
Dockerfile Path	devops-app/Dockerfile
Instance Type	Free ($0/month)
Port	8080

8.3 Free Tier Note

Render's free tier spins down the app after 15 minutes of inactivity. The first request after inactivity may take up to 50 seconds to respond while the container wakes up. In production, a paid tier would be used for always-on availability.

9. Complete CI/CD Flow

This is what happens from the moment a developer pushes code to the app being live:

Step	Action	Tool	Time
1	Developer pushes code: git push origin main	Git + GitHub	Instant
2	GitHub Actions pipeline triggers automatically	GitHub Actions	~5 seconds
3	Code is checked out from repository	actions/checkout	~5 seconds
4	Java 17 is set up on runner	actions/setup-java	~10 seconds
5	Maven compiles Java source code	Maven	~15 seconds
6	JUnit tests run — 3 API tests verified	Maven + JUnit	~20 seconds
7	JAR file is packaged and uploaded as artifact	Maven	~10 seconds
8	Docker image built from Dockerfile	Docker	~30 seconds
9	Image pushed to Docker Hub registry	Docker Hub	~20 seconds
10	Render detects new commit, pulls image	Render	~3-5 minutes
11	Live URL updated with new changes	Render	Complete!

10. Challenges Faced & Solutions

Challenge	Problem	Solution
Jenkins Setup	Required dedicated server, complex configuration	Switched to GitHub Actions — cloud-hosted, zero maintenance
Maven Not Found in CI	Pipeline couldn't find mvn command	Set working-directory: devops-app in YAML

Selenium in CI	No browser available in GitHub Actions runner	Excluded with -Dtest=!SeleniumTest flag
Docker Image Size	Initial image was too large (~500MB)	Used multi-stage build — reduced to ~200MB
CORS Error	Delete button not working on live URL	Added @CrossOrigin(origins="*") to controller
GitHub Token Permissions	Could not push workflow file changes	Updated Personal Access Token to include workflow scope
Port Already in Use	Port 8080 taken when running locally	Stopped existing Docker container before running new one
pom.xml Dependency	spring-boot-starter-webmvc instead of web	Fixed to spring-boot-starter-web for MockMvc support

11. Future Improvements

Improvement	Description	Benefit
Database Integration	Add MySQL/PostgreSQL for persistent storage	Tasks survive app restart
Headless Selenium in CI	Configure headless Chrome in GitHub Actions	UI tests run in CI automatically
SonarQube	Static code analysis for code quality	Better code quality and security
Kubernetes	Container orchestration for scaling	Handle high traffic automatically
Prometheus + Grafana	Monitoring and metrics dashboard	Real-time app health visibility
Environment Separation	Dev, Staging, Production environments	Safer deployments
Docker Compose	Multi-container setup with database	Easier local development

12. Key DevOps Concepts Demonstrated

Concept	Definition	How Used in Project
CI (Continuous Integration)	Auto build and test on every code push	GitHub Actions runs Maven build + JUnit tests
CD (Continuous Deployment)	Auto deploy to production after CI passes	Render auto-redeploys on every push
Containerization	Package app + dependencies into portable container	Docker image contains app + Java runtime
Infrastructure as Code	Define infrastructure in code files	Dockerfile + ci.yml define entire infrastructure
Fail Fast	Stop pipeline immediately on failure	Docker job only runs if build+test passes
Immutable Deployment	Each deploy creates fresh new container	Render pulls new Docker image each time

13. Summary

This project successfully demonstrates a complete, production-grade DevOps pipeline. Starting from a Spring Boot application with REST API and UI, the project implements:

- Automated build and test pipeline using GitHub Actions and Maven
- Two levels of testing — JUnit for API layer and Selenium for UI layer
- Docker containerization with multi-stage build optimization
- Automated image publishing to Docker Hub on every successful build
- Live cloud deployment on Render with auto-redeploy on every push

The entire delivery pipeline is automated — a developer only needs to push code to GitHub, and within 5-10 minutes their changes are live on the internet. This is the core principle of modern DevOps.